

Lagovia Train Tracker

Digital Product School Engineering Track Technical Challenge

Background

Welcome to Lagovia, a small fictional land where the trains are always late and citizens want to know exactly *how* late. Your job is to build a tool that tells them. Conveniently, the rail operator of neighbouring Belgium runs a free, open, no-authentication API that we'll use as our data source: iRail (<https://docs.irail.be/>)

The user story

A user types part of a station name (e.g. "Bru" or "Aac") into a search field or runs an api call. The application returns all upcoming departures from every station whose name contains that substring. For each departure we want to see: train number, destination, scheduled departure time, and current delay in minutes.

You may choose one of two tracks:

Track A: Backend only. Build an HTTP endpoint that returns train departures for a station search.

Track B: Frontend / Fullstack. Build a web page with train delays overview.

Requirements (both tracks)

1. Input contract. Queries shorter than 3 characters must return an explicit error response indicating the input is incomplete. Define and document the shape.
2. Departure window. Return only departures scheduled within the next 15 minutes from "now."
3. **Track A (Backend):** Expose a single endpoint, e.g. `GET /departures?q=Bru`. Return JSON. Define the shape yourself and document it.

Track B — Frontend: A search input that triggers a search after the user types ≥ 3 characters. A clear list or table of departures, grouped or labelled by station.

Tech stack

Any language and framework. Our preference (mirrors our environment) is:

Backend: Node.js (Express) or Python (FastAPI)

Frontend: React

Choose something else if you have good reason; explain why in the README.

Submission: public Git repository (GitHub/GitLab)

README must include:

- How to install and run locally.
- Decisions, trade-offs, and known limitations.
- Roughly how long you spent.
- AI usage report (in the README or a separate `AI_USAGE.md`):

Using AI is allowed and expected. Please include the following in your report:

- Which AI tools you used and for what.
- Public links to chat conversations where feasible, or a representative selection.
- Any implementation plans or prompts you used.
- Briefly: what you accepted as-is, what you rewrote, what you rejected.

Time budget

You have one week to submit from the day you receive this brief.

Walkthrough. Expect a follow-up session where we'll ask you to talk through your code, explain why you made certain choices, and make a small live change or fix. If you copy-pasted something without understanding it, this is where it will show. Use AI freely while building, but make sure you own the result.

Bonus (do not pursue at the cost of the basics)

Fuzzy search in addition to substring (e.g. `"Antwerpen"` finds `Antwerpen-Centraal`).

Good luck. The trains are waiting (well, not waiting. Late.)